ADVANCED SOFTWARE DEVELOPMENT AND DEVOPS (ASDD)

PRACTICAL FILE

Experiments 1 to 8

Agile • Git • Jenkins • CI/CD • Kubernetes • Docker • Grafana • AWS Lambda

Name	
Roll No.	
Class / Division	
Subject Teacher	

Department of Computer Engineering

INDEX

Exp. No.	Title of the Experiment	Page No.
1	Agile Software Project Lifecycle using Jira with DevOps Pipeline Integration	
2	Version Control Operations using Git and Collaboration Workflows	
3	Installation of Jenkins and Sample Build	
4	Integrate Jenkins with GitHub and Simulate a CI/CD Pipeline	
5	Kubernetes – Deploy, Expose and Scale an Application	
6	Docker – Develop with Containers	
7	Monitoring using Grafana	
8	Serverless Function using AWS Lambda	

EXPERIMENT 1

Agile Software Project Lifecycle using Jira with DevOps Pipeline Integration

Aim

To study and implement the Agile software project lifecycle using Jira with DevOps pipeline integration.

Theory

- Planning: Teams use Jira's Agile features to plan work, including Scrum and Kanban boards, backlogs and sprints, providing a single source of truth for development and operations.
- Sprint Planning: The team decides what to work on and how much to commit to, setting clear sprint goals and planning the workload realistically based on past velocity.
- Daily Stand-Up: A quick check-in covering what was done yesterday, what is planned for today and any blockers, keeping the team aligned and focused.
- Mid-Sprint Review: A checkpoint to assess progress, identify blockers and decide whether help is needed, which is important in cross-functional DevOps teams.
- Continuous Feedback: Feedback is collected organically and structurally to assess the impact of each release on user experience and to inform future enhancements.
- Integration with CI/CD: Jira integrates with CI/CD pipelines to automate build, testing and deployment, making the lifecycle streamlined and efficient.

Agile project management helps teams develop products efficiently while adapting to changing requirements. Jira turns Agile principles such as iterative planning and real-time visibility into daily workflows, supporting Scrum or Kanban within a customizable platform.

Procedure – Setting up a Jira account

1. Go to the Jira website and click Sign Up. Sign up using an email address or a Google account.
2. Choose a plan. Jira offers several plans, including a free version for smaller teams.
3. Complete the account setup by following the on-screen instructions, including the workspace name and basic settings.

Creating the first project

4. After logging in, open the Jira dashboard and click Create Project.
5. Select an Agile template: Scrum for iterative sprints or Kanban for continuous flow work.
6. Name the project so that it reflects its scope or the product being developed.
7. Set permissions to control who can access and modify the project.

Basic Jira terminology

- Story: A small unit of work, usually a feature or task.
- Sprint: A fixed period (typically 1–4 weeks) in which a set of stories or tasks is completed.
- Epic: A large body of work that can be broken down into smaller stories.
- Backlog: A prioritised list of all tasks that need to be completed in the future.

Navigating the Jira interface

- Projects panel: quick access to all active projects.
- Issues and filters: locate tasks using built-in or custom filters.
- Agile boards: navigate directly to Scrum or Kanban boards.
- Reports: access charts and metrics such as burndown, sprint and velocity reports.

Each element of the dashboard is customizable so that teams can configure it around sprint progress, team workloads or upcoming deadlines.

Creating and managing backlogs

8. Add new issues with Create Issue (user story, task or bug).
9. Set priorities by dragging and dropping tasks to reorder them.
10. Move items from the backlog into a sprint for execution.
11. Use labels for categorization and story points for effort estimation.

Tracking sprints

12. Create a sprint from the backlog view.
13. Drag tasks from the backlog into the new sprint.
14. Start the sprint by setting a duration (typically 1–4 weeks) and a name.
15. Monitor progress in the Active Sprint view to track task movement and sprint goals.

Managing tasks and workflows

16. Click Create and select the project and issue type.
17. Story: a feature or functionality from the user's perspective.
18. Task: a general to-do item.
19. Bug: a defect or problem in the product.
20. Fill in the summary, description, priority and due date.
21. Use labels and components to categorize and filter issues later.
22. Assign the issue to a team member using the Assignee field.
23. Set a priority level (Highest, High, Medium, Low) to indicate urgency.
24. Add the issue to a sprint or the backlog depending on when it is expected to be completed.

Tracking progress

- Use the Active Sprint view or Kanban board to see whether each issue is To Do, In Progress, Review or Done.
- Team members drag issues between columns, updating status in real time.
- Status colours and avatars identify blocked issues, unassigned tasks or recently updated items at a glance.

Best practices

- Keep tasks clear, concise and actionable.
- Use labels, components and epics to organise work.
- Groom the backlog regularly to maintain priority and focus.

- Limit work-in-progress on boards to avoid overloading the team.

Conclusion

The Agile software project lifecycle was studied and implemented using Jira with DevOps pipeline integration.

EXPERIMENT 2

Version Control Operations using Git and Collaboration Workflows

Aim

To perform version control operations using Git and apply collaboration workflows for software development.

Prerequisite

Git installed on the system and a working GitHub account.

Procedure

1. Verify that Git is installed and configure the global identity:

```
git --version
git config --global user.name "<your-name>"
git config --global user.email "<your-email>"
git config --list
```

2. Create a project folder and initialise a Git repository:

```
mkdir GitProject
cd GitProject
git init
git status
```

3. Create the first file and commit it:

```
echo "# GitProject" > README.md
git add README.md
git commit -m "Initial Commit"
```

4. Rename the branch to main and connect the local repository to GitHub:

```
git branch -M main
git remote add origin <GitHub repository link>
git push -u origin main
```

5. Collaboration workflow: create a feature branch, commit changes, push the branch, open a pull request, obtain review and merge it into main.

```
git checkout -b feature-branch
git add .
git commit -m "Add feature"
git push origin feature-branch
```

Commands and Output

The commands above were executed successfully. The repository was created locally, connected to GitHub and pushed to the remote main branch.

Conclusion

Version control operations using Git were performed and the collaboration workflow was applied successfully.

EXPERIMENT 3

Installation of Jenkins and Sample Build

Aim

To install Jenkins, launch it on an available port and trigger a sample build for the built-in "Hello World" program.

Prerequisite

Java (JDK) must be installed on the system before Jenkins is installed.

Procedure

25. Verify the Java installation and version.
26. Download Jenkins (WAR file or the platform installer) from the official Jenkins website.
27. Start Jenkins and connect it to any available port.
28. Open Jenkins in the browser, unlock it with the initial administrator password and install the suggested plugins.
29. Create the administrator user and complete the setup.
30. Create a Freestyle project and add a build step that prints "Hello World".
31. Trigger the build and verify the output in the console.

Commands

```
java -version
java -jar jenkins.war --httpPort=8080
```

Output

Jenkins was launched successfully in the browser. A sample "Hello World" build was triggered and completed successfully, and the console output confirmed the expected message.

Conclusion

Jenkins was installed, launched on an available port and a sample "Hello World" build was triggered successfully.

EXPERIMENT 4

Integrate Jenkins with GitHub and Simulate a CI/CD Pipeline

Aim

To integrate Jenkins with GitHub and simulate a CI/CD pipeline for automated building and testing of a software project.

Prerequisite

A working Jenkins installation and a GitHub login.

Procedure

32. Install the Git and GitHub plugins in Jenkins (Manage Jenkins → Plugins).
33. Create a repository on GitHub and push the project source code to it.
34. Add a Jenkinsfile to the repository that defines the pipeline stages (Checkout, Build, Test, Deploy).
35. In Jenkins, add a GitHub credential (Personal Access Token) under Manage Jenkins → Credentials.
36. Create a Pipeline job, select Pipeline script from SCM, choose Git, enter the repository URL, select the credential and the main branch, and specify the Jenkinsfile path.
37. Configure a build trigger: GitHub webhook or Poll SCM, so that a push starts the pipeline automatically.
38. Run the pipeline and verify each stage in the console output / Blue Ocean view.

Sample Jenkinsfile

```
pipeline {
    agent any
    stages {
        stage('Checkout') { steps { git '<repository-url>' } }
        stage('Build')    { steps { echo 'Building...' } }
        stage('Test')     { steps { echo 'Running tests...' } }
        stage('Deploy')   { steps { echo 'Simulated deployment...' } }
    }
}
```

Output

A pipeline was created in Jenkins and triggered from a GitHub change. All stages completed successfully, simulating an automated build, test and deployment workflow.

Conclusion

Jenkins was integrated with GitHub and a CI/CD pipeline for automated building and testing was simulated successfully.

EXPERIMENT 5

Kubernetes – Deploy, Expose and Scale an Application

Aim

To deploy, expose and scale an application on a Kubernetes cluster.

Choose one option

Option A – Minikube already installed on the PC (Docker must be running). Open PowerShell and run the commands below, starting with minikube start --driver=docker.

Option B – No installation needed (works on any PC, in the browser):

39. Open https://killercoda.com and log in (GitHub login can be used).
40. Click Playgrounds → Kubernetes (the 1-node one) → Start.
41. A Linux terminal opens in the browser. Skip minikube start because the cluster is already running.
42. Run the commands below.

Commands

```
kubectl get nodes
kubectl create deployment nginx --image=nginx
kubectl get deployments
kubectl get pods
kubectl expose deployment nginx --type=NodePort --port=80
kubectl get svc
# Option A: minikube service nginx
# Option B: curl localhost:<NodePort>
kubectl scale deployment nginx --replicas=3
kubectl get pods
kubectl describe deployment nginx
kubectl delete service nginx
kubectl delete deployment nginx
```

Output

The node was reported as Ready. The nginx deployment was created, exposed using a NodePort service and tested in the browser / with curl, which printed "Welcome to nginx!". The deployment was then scaled to 3 replicas and all three pods were shown as Running. Finally, the service and deployment were deleted.

Conclusion

nginx was deployed, exposed, scaled to 3 replicas and deleted on Kubernetes.

EXPERIMENT 6

Docker – Develop with Containers

Aim

To develop and run an application using Docker containers.

Procedure

To get started, either clone or download the project as a ZIP file to the local machine.

```
git clone https://github.com/docker/getting-started-todo-app
```

After the project is cloned, navigate into the new directory created by the clone:

```
cd getting-started-todo-app
```

Once the project is available, start the development environment using Docker Compose. To start the project using the CLI, run:

```
docker compose watch
```

The container is now visible in Docker Desktop.

```
docker build -t welcome .
```

Output

The project was cloned and started with Docker Compose. The running container was verified in Docker Desktop, and the image was built successfully using the docker build command.

Conclusion

An application was developed and run using Docker containers and Docker Compose.

EXPERIMENT 7

Monitoring using Grafana

Aim

To monitor system and application metrics using Grafana dashboards.

Introduction

Grafana is an open-source analytics and monitoring platform designed to visualise time-series data through interactive and customisable dashboards. It connects to various data sources such as databases, cloud services and monitoring systems, transforming raw data into meaningful insights using graphs, charts and alerts.

Grafana is used to monitor infrastructure performance, application health and real-time system metrics, including server uptime, network activity and resource utilisation. It is commonly integrated with monitoring tools such as Prometheus, InfluxDB or Elasticsearch, which allows real-time tracking of system performance and proactive issue detection. Alerts can be configured to trigger when a metric exceeds a defined threshold, helping to prevent downtime. Grafana is cross-platform and supports Linux, Windows and macOS.

How to download Grafana

43. Go to the official Grafana download website.
44. Select the Windows version.
45. Download and save the .msi installer file.

How to install Grafana

46. Locate the .msi installer file that was downloaded.
47. Double-click to launch the installer.
48. Follow the setup wizard instructions.
49. Click Install and wait for the installation to complete.

```
net start Grafana
```

How to set up Grafana

50. Launch Grafana from the Start menu.
51. Open a web browser and go to http://localhost:3000.
52. Log in with the default credentials (username: admin, password: admin) and update the password.
53. Add and configure data sources.
54. Create dashboards.

How to use the Grafana dashboard

55. Open Grafana in a browser and log in with the credentials.
56. Click the + Create icon, select Dashboard and then Add new panel.
57. Choose a connected data source, write a query using PromQL, SQL or another supported language, and select the visualisation type (graph, gauge, table).

58. Customise the visualisation and layout: adjust axes, labels and thresholds; drag and resize panels; group related metrics.
59. Apply configuration options and set alerts under Alerting → Alert rules by defining conditions (e.g. CPU > 80%) and choosing notification channels (email, Slack, webhook).
60. Save the dashboard and share it via a link or the export options.

Conclusion

System and application metrics were visualised using Grafana dashboards, and alert rules were configured successfully.

EXPERIMENT 8

Serverless Function using AWS Lambda

Aim

To create and deploy a serverless function using AWS Lambda and execute it on a schedule.

Procedure

61. Sign in to the AWS Management Console and open the Lambda service.
62. Click Create a function.
63. Choose Author from scratch, enter the function name (for example, myStopinato), select a Python runtime and create the function.
64. Add a trigger: choose EventBridge (CloudWatch Events), create a new rule, name it everyMinute and use the schedule expression rate(1 minute).
65. Open the Code tab and write the function using the boto3 library to stop the required EC2 instances.
66. Click Deploy to deploy the code. Select or create a test event and run Test.
67. Open the Monitor tab to view invocations, duration, error count and CloudWatch logs.

Function code

```
import boto3

region = 'us-east-1'
instances = ['<REPLACE_WITH_INSTANCE_ID>']
ec2 = boto3.client('ec2', region_name=region)

def lambda_handler(event, context):
    ec2.stop_instances(InstanceIds=instances)
    print('stopped your instances: ' + str(instances))
```

Output

The Lambda function was created, configured with an EventBridge schedule trigger and deployed successfully. The Monitor tab confirmed invocations and the CloudWatch logs recorded the function output.

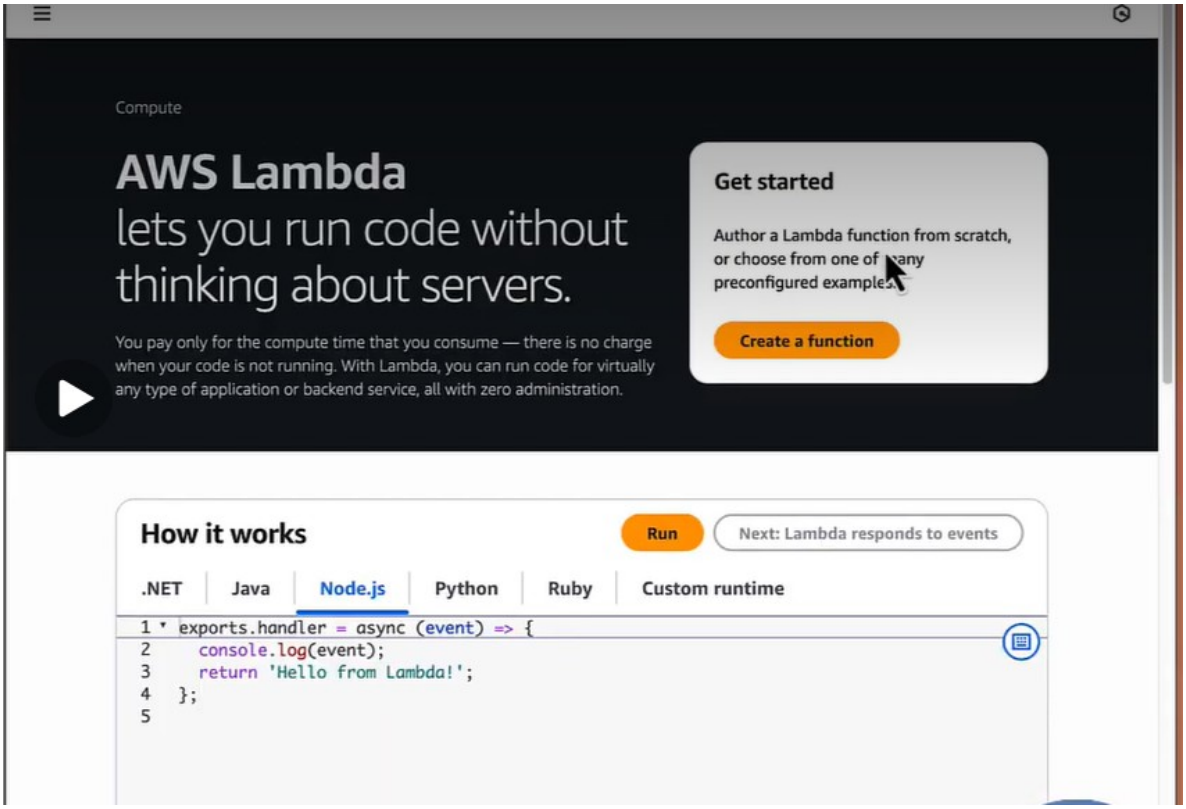


Figure 8.1 – AWS Lambda service home page

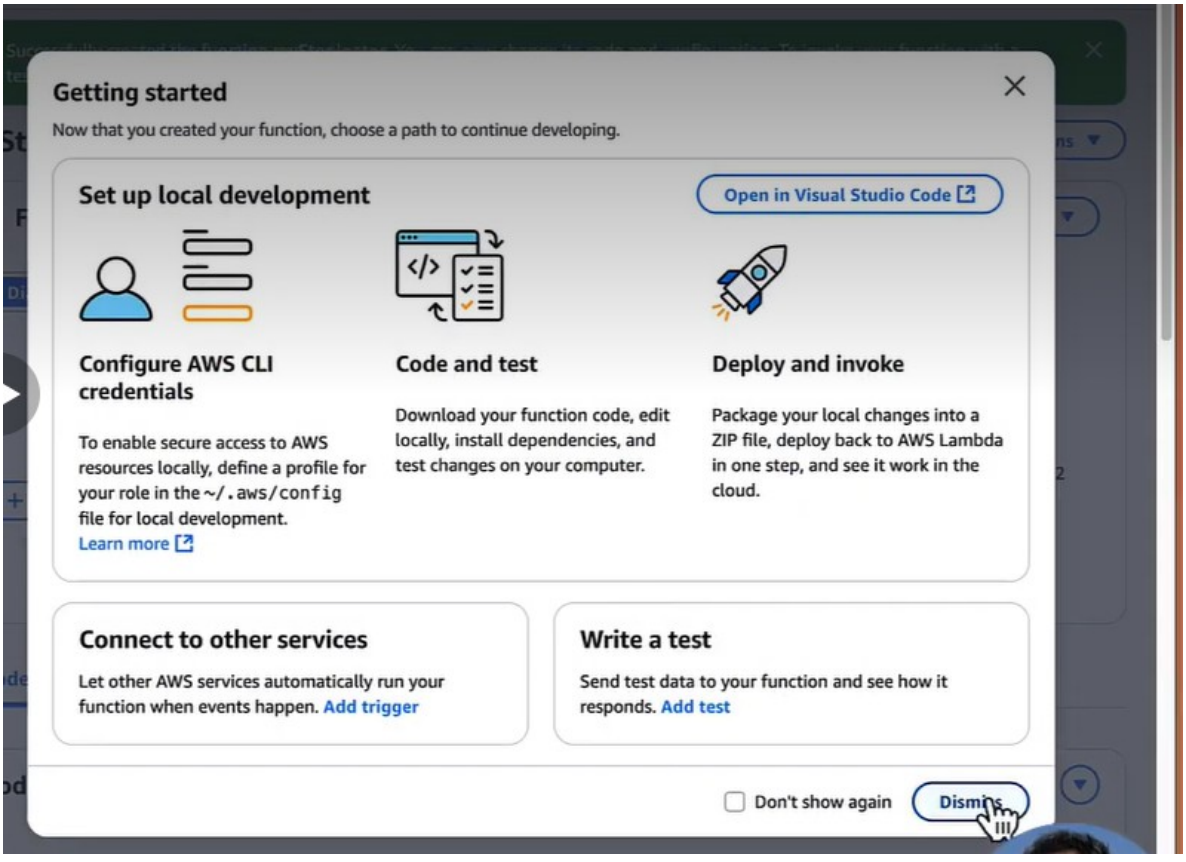


Figure 8.2 – Getting started options after creating the function

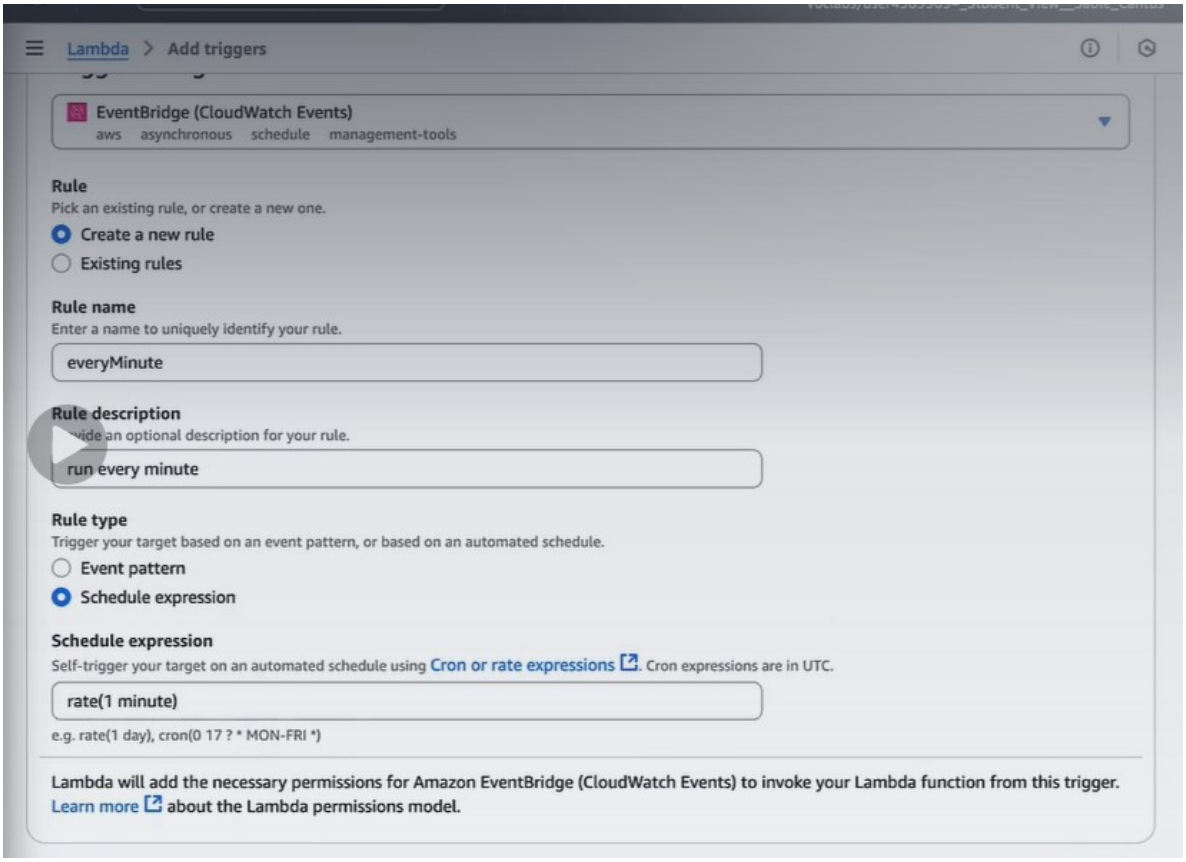


Figure 8.3 – Adding the EventBridge trigger with rate(1 minute)

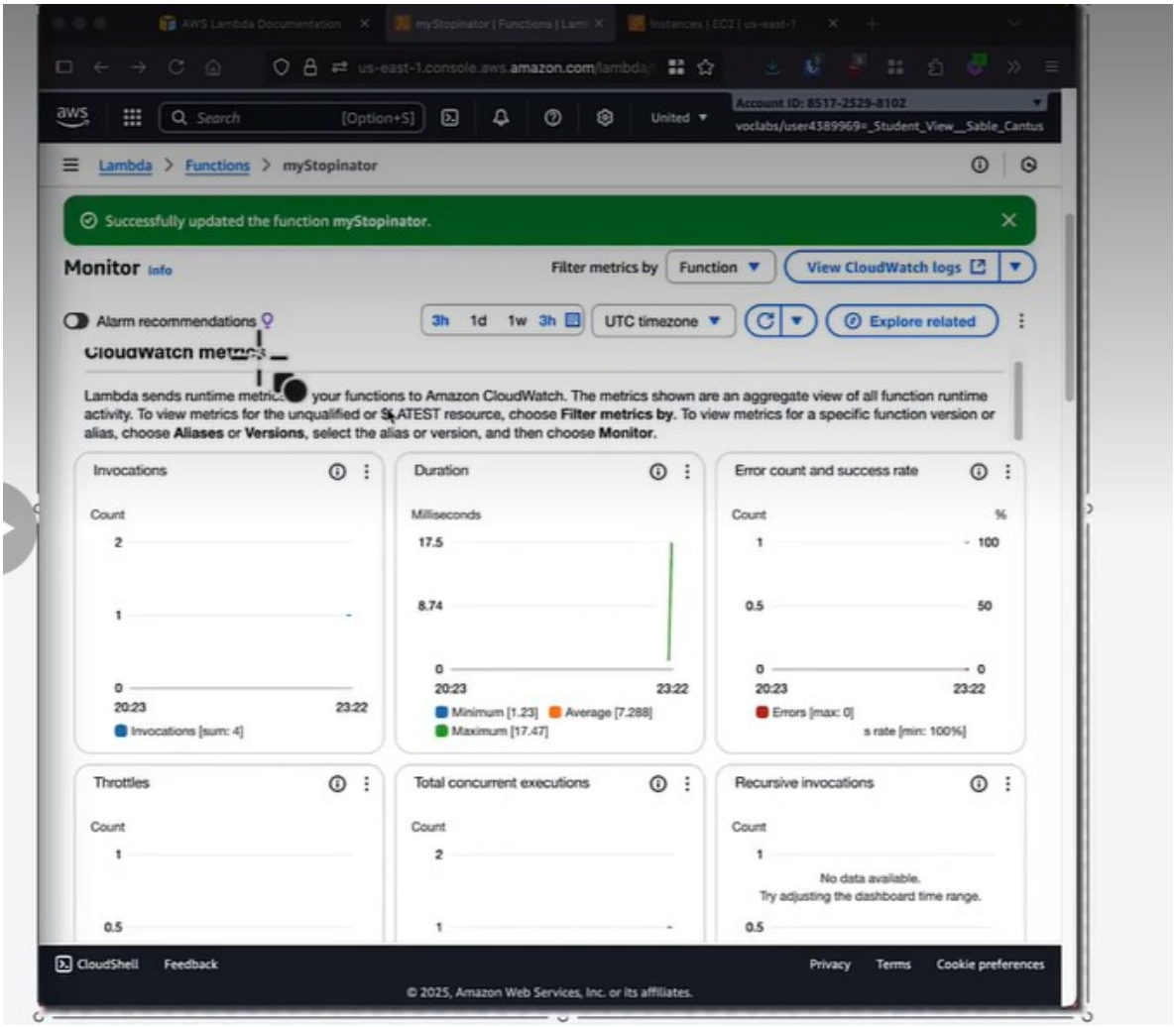


Figure 8.4 – Monitor tab showing invocation and duration metrics

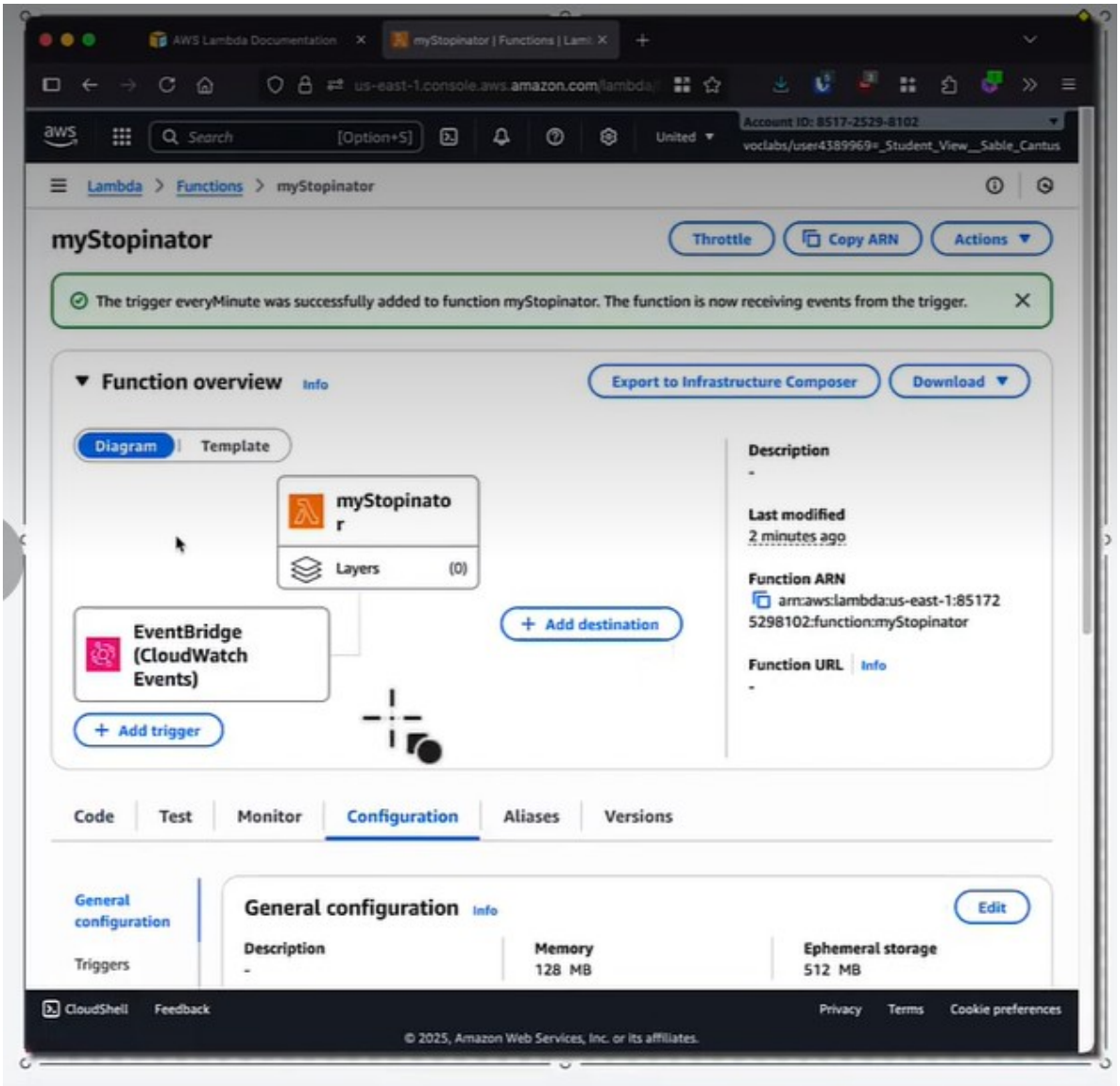


Figure 8.5 – Function overview with the EventBridge trigger attached

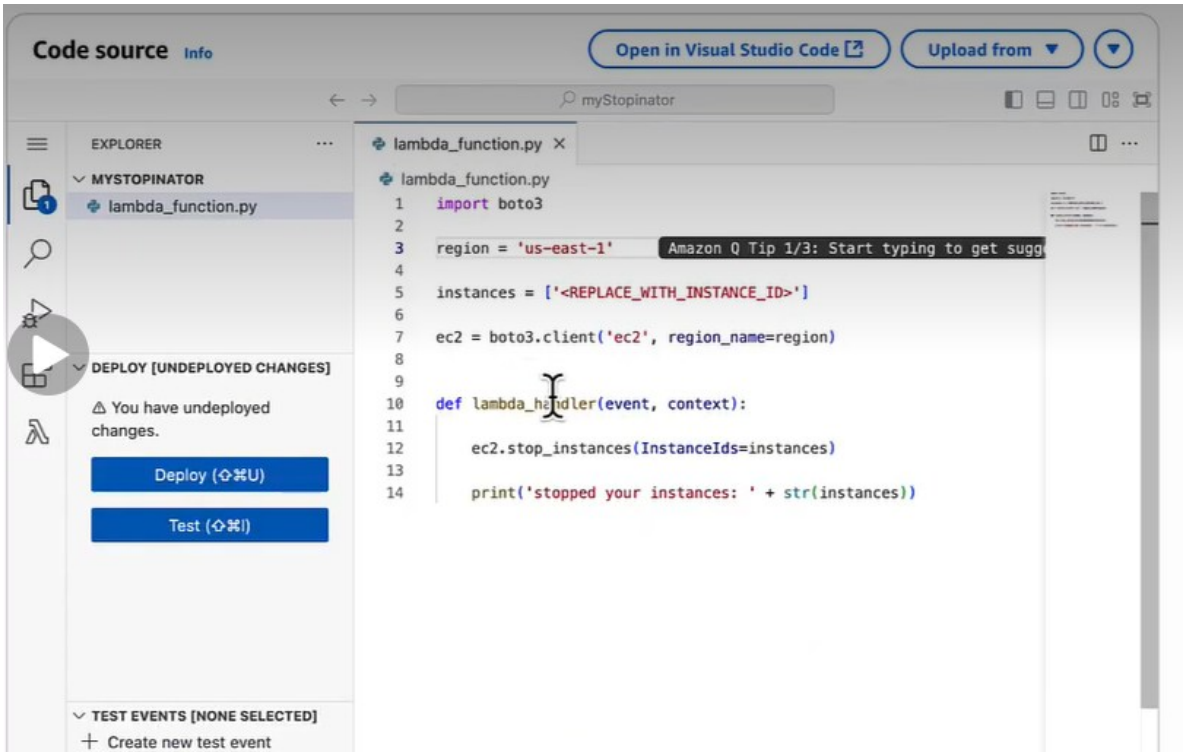


Figure 8.6 – Function code in the Code source editor

Conclusion

A serverless function was created and deployed using AWS Lambda, triggered on a schedule through EventBridge and monitored using CloudWatch metrics.